

به نام خدا

پاسخ بخش تئوری تمرین دوم درس پردازش زبان طبیعی

استاد درس: دکتر یاسمن برشبان

علی افشار دگرسی ۱۴۰۴۱۲۲۶۱۵۱۰۱

سوال اول: توجه

در مکانیزم توجه (Attention Mechanism) معمولاً در هر گام از decoder ، با ترکیب وزن دار بردارهای نهان تمام گامهای encoder یک بردار context vector ساخته می شود. سپس خروجی بعدی decoder با استفاده از آن بردار context vector محاسبه می شود. این فرآیند به صورت زیر تعریف می شود:

$$a_t(s) = \text{align}(h_t, \bar{h}_s) = \frac{\exp(\text{score}(h_t, \bar{h}_s))}{\sum_{s'} \exp(\text{score}(h_t, \bar{h}_{s'}))}$$

$$c_t = \sum_s a_t(s) \bar{h}_s$$

$$\tilde{h}_t = \tanh W_c[c_t, h_t]$$

$$p(y_t | y_{<t}, x) = \text{softmax}(W_s \tilde{h}_t)$$

الف) چه تعبیری از مقادیر $a_t(s)$ و c_t می توان داشت.

پاسخ:

$a_t(s)$: ضریب هر hidden unit می باشد که مقداری بین ۰ تا ۱ می باشد و مجموع ضرایب تمام hidden unit ها برابر ۱ می شود. هر hidden unit نشان دهنده اطلاعات مربوط به ورودی بخش encoder در موقعیت s است. این ضرایب نشان دهنده تاثیر اطلاعات غنی شده این توکن ها در بخش encoder در خروجی توکن موقعیت t در decoder می باشد. پس می شود اینطور بیان کرد که در هر گام از decoder مقدار وزن ها برای ترکیب بردارهای نهان تمام گام های encoder اینگونه مشخص می شود. به تعبیر دیگر، خود decoder انتخاب میکند که هر گام یا بخش encoder با چه ضریبی در تولید خروجی هر گام خود تاثیر داشته باشد. بطور خلاصه اهمیت موقعیت s در encoder برای گام t در decoder می باشد.

c_t : همان مجموع یا ترکیب وزن دار گام های نهان بخش encoder می باشد که برای هر گام encoder منحصر به فرد می باشد ولی طول یکسانی دارد. همان context vector که در صورت سوال ذکر شده است می باشد. بطور خلاصه، خلاصه وزنی جمله مبدا می باشد.

ب) دو تابع متداولی که برای $score(h_t, \bar{h}_s)$ استفاده می شود را بیان کنید.

پاسخ:

تابع اول: ضرب داخلی دو بردار، فرض می کند که فضای حالت های پنهان در encoder و decoder هم تراز هستند.

تابع دوم: استفاده از شبکه عصبی تک لایه به شکل زیر:

$$score(h_t, \bar{h}_s) = v_a^T \tanh(W_a[h_t; \bar{h}_s])$$

که W و v پارامتر یا وزن هایی هستند که در طول آموزش تنظیم می شوند.

سوال دوم: Transformer

الف) چرا در مدل Transformer نیازی به RNN نیست؟ دو مزیت مهم Transformer نسبت به RNN را توضیح دهید.

پاسخ:

مدل Transformer، اساساً بر پایه مکانیسم‌های Attention (به ویژه Self-Attention) و Feed-Forward Networks ساخته شده است پس این مدل نیازی به RNN ندارد.

دو مزیت مهم Transformer نسبت به RNN:

۱. سرعت آموزش و استنتاج بالاتر (Parallelization): در RNN‌ها به دلیل پردازش بصورت دنباله‌ای، نمی‌توانند به طور کامل از سخت‌افزارهای موازی مانند GPU بهره ببرند، که باعث می‌شود آموزش مدل‌های بزرگ زمان‌بر باشد. در مقابل، Transformer تمام توکن‌ها را همزمان پردازش می‌کند، که آموزش را تا چندین برابر سریع‌تر می‌کند.
۲. مدیریت بهتر وابستگی‌های بلندمدت (Long-Range Dependencies): در RNN‌ها (حتی با LSTM یا GRU) اغلب با مشکل vanishing/exploding gradients مواجه هستند، که باعث می‌شود وابستگی‌های بین عناصر دور از هم (مانند کلمات ابتدای و انتهای یک جمله طولانی) به خوبی یادگیری نشود. Transformer با مکانیسم Attention، مستقیماً به همه موقعیت‌ها دسترسی دارد و می‌تواند روابط بین هر جفت عنصر را محاسبه کند، بدون اینکه اطلاعات از لایه‌های متعدد عبور کند.

ب) مفهوم Self-Attention را توضیح دهید. چرا Self-Attention باعث میشود ارتباط بین کلمات دور از هم بهتر مدل شود؟

پاسخ: Self-Attention (یا توجه به خود) یک مکانیسم کلیدی در مدل‌های مبتنی بر Transformer است. این مکانیسم به مدل اجازه می‌دهد تا در یک دنباله ورودی (مانند یک جمله یا توالی داده‌ها)، هر عنصر را با توجه به روابطش با سایر عناصر در همان دنباله، وزن‌دهی کند. به عبارت ساده‌تر، Self-Attention به مدل کمک می‌کند تا بفهمد کدام قسمت‌های ورودی برای پیش‌بینی یا درک یک قسمت خاص مهم‌تر هستند. در مدل‌های سنتی مانند RNN (شبکه‌های عصبی بازگشتی) یا LSTM، اطلاعات به صورت sequential (ترتیبی) پردازش می‌شود یعنی هر گام وابسته به گام قبلی است. این امر باعث مشکلاتی مثل Vanishing Gradient Problem و وابستگی به فاصله می‌شود. اما Self-Attention این مشکلات را حل می‌کند چون هر توکن مستقیماً به همه توکن‌های دیگر در دنباله دسترسی دارد، بدون نیاز به اینکه از توکن‌های میانی عبور کند.

پ) نقش Positional Encoding در Transformer چیست و اگر حذف شود چه مشکلی ایجاد میشود؟

پاسخ: Positional Encoding یک بردار عددی است که به embedding هر توکن اضافه می‌شود. این بردار بر اساس موقعیت (position) توکن در ورودی محاسبه می‌شود و به مدل کمک می‌کند تا ترتیب کلمات را درک کند. بدون Positional Encoding، مدل Transformer مانند یک "کیسه کلمات" (bag-of-words) عمل می‌کند و اطلاعات ترتیب را از دست می‌دهد. به عنوان مثال مدل نمی‌تواند تفاوت بین جملات با ترتیب متفاوت اما کلمات یکسان را بفهمد. مثلاً "سگ گربه را تعقیب کرد" و "گربه سگ را تعقیب کرد" برای مدل یکسان به نظر می‌رسند، که منجر به

خطاهای فاحش در خروجی می‌شود. همچنین در تولید یک جمله منسجم در خروجی برای وظایف وابسته به ترتیب نیز عملکرد ضعیفی خواهد داشت.

سوال سوم: LLM

الف) تفاوت کلی بین مدل‌های Encoder-Only (مثل BERT) و Decoder-Only (مثل GPT) را توضیح دهید. هرکدام برای چه کاربردهایی مناسب تر هستند؟
پاسخ:

: Encoder-Only

این مدل‌ها فقط از بخش encoder معماری Transformer استفاده می‌کنند. encoder مسئول پردازش ورودی به صورت دوطرفه (bidirectional) است، یعنی مدل به تمام متن ورودی (از چپ به راست و راست به چپ) همزمان دسترسی دارد و می‌تواند زمینه کامل را درک کند.

کاربردهای مناسب‌تر:

- طبقه‌بندی متن مثل تحلیل احساسات sentiment analysis
- استخراج اطلاعات مثل شناسایی موجودیت NER
- پاسخ به سوالات QA وقتی ورودی بصورت کامل داده میشود
- در بخش ابتدایی auto-encoder ها برای استخراج ویژگی های غنی یا embedding

: Decoder-Only

این مدل‌ها فقط از بخش decoder معماری Transformer استفاده می‌کنند. decoder به صورت یک طرفه کار می‌کند، یعنی متن را کلمه به کلمه تولید می‌کند و هر کلمه جدید بر اساس کلمات قبلی پیش‌بینی می‌شود.

کاربردهای مناسب تر:

- تولید متن
- ترجمه ماشین یا خلاصه سازی
- تکمیل متن

به طور خلاصه، Encoder-Only بر "درک" تمرکز دارد، در حالی که Decoder-Only بر "تولید" تأکید می کند.

ب) منظور از Hallucination در LLM ها چیست؟ چرا رخ میدهد و چگونه میتوان آن را کاهش داد؟

پاسخ: Hallucination (یا توهم زایی) در LLM ها به پدیده‌ای اشاره دارد که مدل اطلاعاتی را تولید می کند که نادرست، ساختگی یا بدون پایه واقعی است، اما آن را با اطمینان کامل به عنوان واقعیت ارائه می دهد.

چرا رخ می دهد؟

LLM ها بر اساس داده‌های عظیم آموزش دیده‌اند، اما مکانیسم کارشان آماری است و نه مبتنی بر درک واقعی جهان. دلایل اصلی عبارتند از:

- آموزش بر اساس الگوهای آماری: مدل‌ها متن را بر اساس احتمال وقوع کلمات بعدی پیش‌بینی می کنند. اگر داده‌های آموزشی ناقص یا قدیمی باشند، مدل ممکن است اطلاعات را "پر کند" و محتوای ساختگی تولید کند.
- کمبود داده یا دانش خارجی: مدل‌ها دانششان محدود به داده‌های آموزشی است و نمی‌توانند به اطلاعات واقعی زمان دسترسی داشته باشند (مگر با ابزارهای خارجی). این باعث می‌شود در موضوعات پیچیده یا نادر، اشتباه کنند.

- تأثیر داده‌های آموزشی: اگر داده‌ها شامل اطلاعات غلط، داستان‌های تخیلی یا محتوای اینترنتی نادرست باشند، مدل ممکن است آن‌ها را به عنوان واقعیت تکرار کند.

چگونه می‌توان Hallucination را کاهش داد؟

- Prompt Engineering: پرامپت‌ها را طوری طراحی کنید که مدل را به تفکر گام‌به‌گام (Chain of Thought) تشویق کند. مثلاً بگویید "ابتدا واقعیت‌ها را بررسی کن، سپس پاسخ بده" تا مدل کمتر به ساختگی‌ها متکی شود.
- استفاده از RAG یا Retrieval-Augmented Generation: مدل را با یک سیستم جستجو ترکیب کنید تا قبل از پاسخ، اطلاعات واقعی را از منابع خارجی (مانند وب یا پایگاه داده) بازیابی کند و بر اساس آن‌ها پاسخ دهد.
- Fine-Tuning: مدل را روی داده‌های خاص و دقیق آموزش دهید تا در حوزه‌های خاص دقت بیشتری داشته باشد.

ممنون از توجهتون